

Machine Learning

1. Unsupervised Learning

1.1 Definition

Unsupervised learning is a type of machine learning where the algorithm learns from unlabeled data without any predefined outputs or target variables.

The algorithm finds hidden patterns, similarities, or groupings within the data to get insights and make data-driven decisions without human intervention.

1.2 Core Concept

Unlike supervised learning, there are no labels. The machine must discover structure by itself.

- Input: Raw, unlabeled data
- Process: Algorithm finds patterns automatically
- Output: Clusters, rules, or reduced-dimension representations

1.3 Why Unsupervised Learning?

- Useful when large datasets make manual labeling impractical or costly.
- Ideal for Exploratory Data Analysis (EDA).
- Automatically discovers similarities and differences in information.

1.4 Real-World Applications

Customer Segmentation

Divides customers based on common characteristics (demographics, behaviour) so marketing teams can target them more effectively.

Example axes: Income vs. Age — customers are grouped into segments based on these features.

Cross-Selling

Cross-selling offers customers products compatible with what they are already purchasing.

Example: A customer who buys a laptop is offered a laptop bag, mouse, or keyboard.

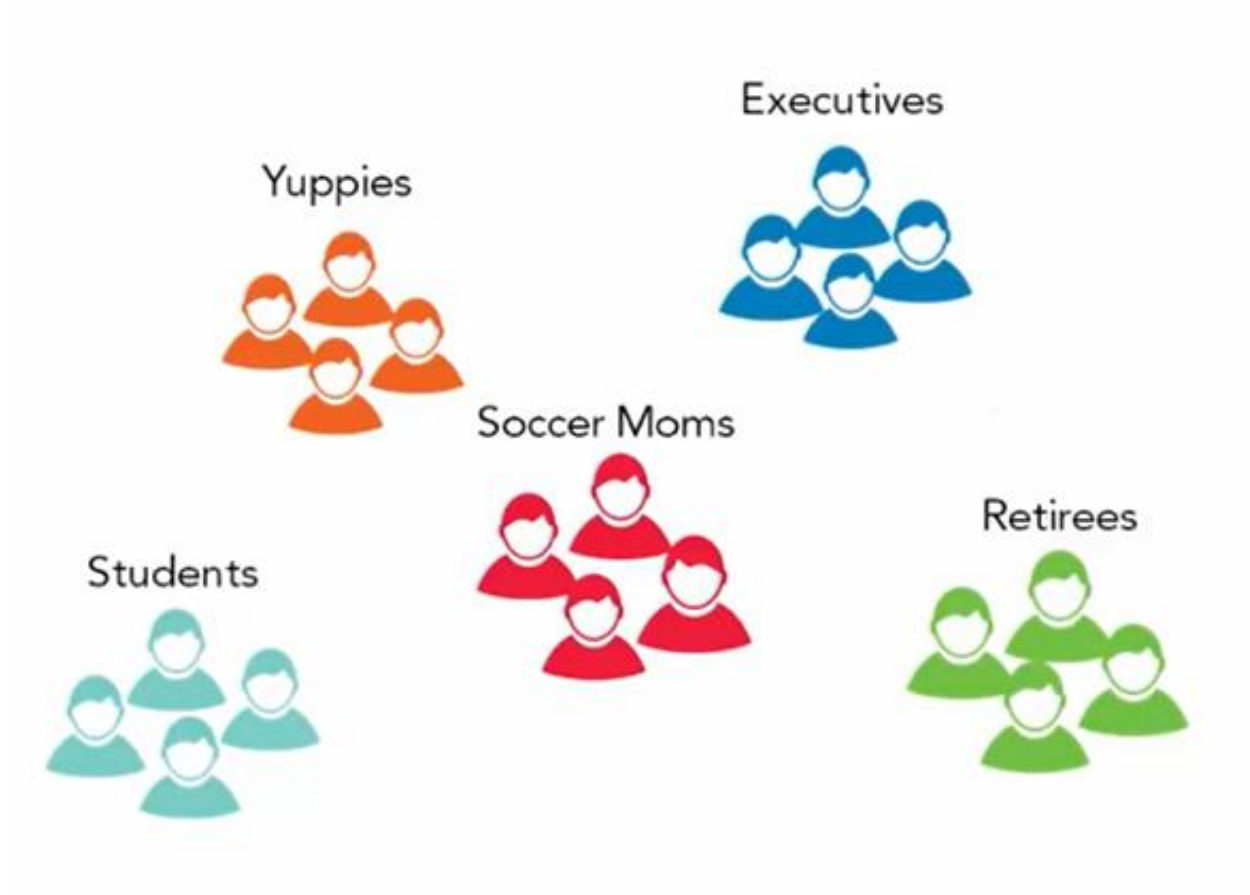


Figure 1.1 — Customer Segmentation by Income and Age

1.5 How Unsupervised Learning Works (Step-by-Step)

1. Start with an unlabeled input dataset (e.g., images of cats and dogs with no labels).
2. Feed the raw data into the ML model. The model does not know what to look for.
3. The model interprets the raw data and finds hidden patterns.
4. A suitable algorithm (e.g., K-Means clustering) is selected and applied.
5. The algorithm divides data objects into groups based on similarities and differences.

Data to conduct Customer Segmentation

Demographic Information

Age

Gender

Marital Status

Income

Transactional Information

Products purchased

\$ volume purchased

of items purchased

Time of purchase

Figure 1.2 — Unsupervised Learning Workflow

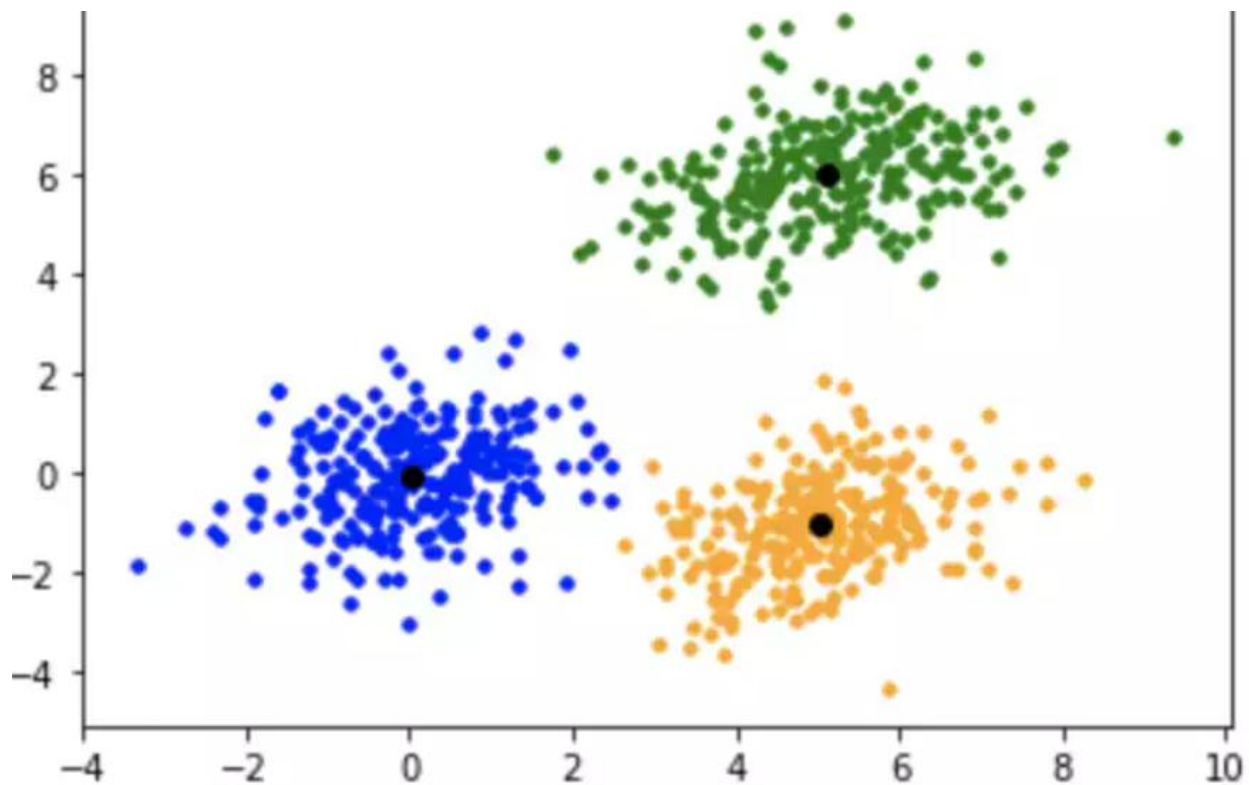
1.6 Types of Unsupervised Learning

- **Clustering**
- **Association Rule Mining**
- **Dimensionality Reduction**

2. Clustering

2.1 Definition

Clustering algorithms interpret input data and find **natural groups** in **feature space**. Data points within the same cluster are more similar to each other than to those in other clusters.



Ideal clustering with a single centroid in each cluster. Source: [GeeksforGeeks](#)

Figure 2.1 — Types of Clustering Algorithms

2.2 Types of Clustering

2.2.1 Exclusive (Hard) Clustering

Each data point belongs to exactly one cluster.

Example: A customer is classified as either 'High Value' or 'Low Value' — not both.

2.2.2 Overlapping (Soft / Fuzzy) Clustering

A data point may belong to multiple clusters simultaneously, with varying degrees of membership.

Example: A customer is 70% 'High Value' and 30% 'Medium Value' at the same time.

2.2.3 Hierarchical Clustering

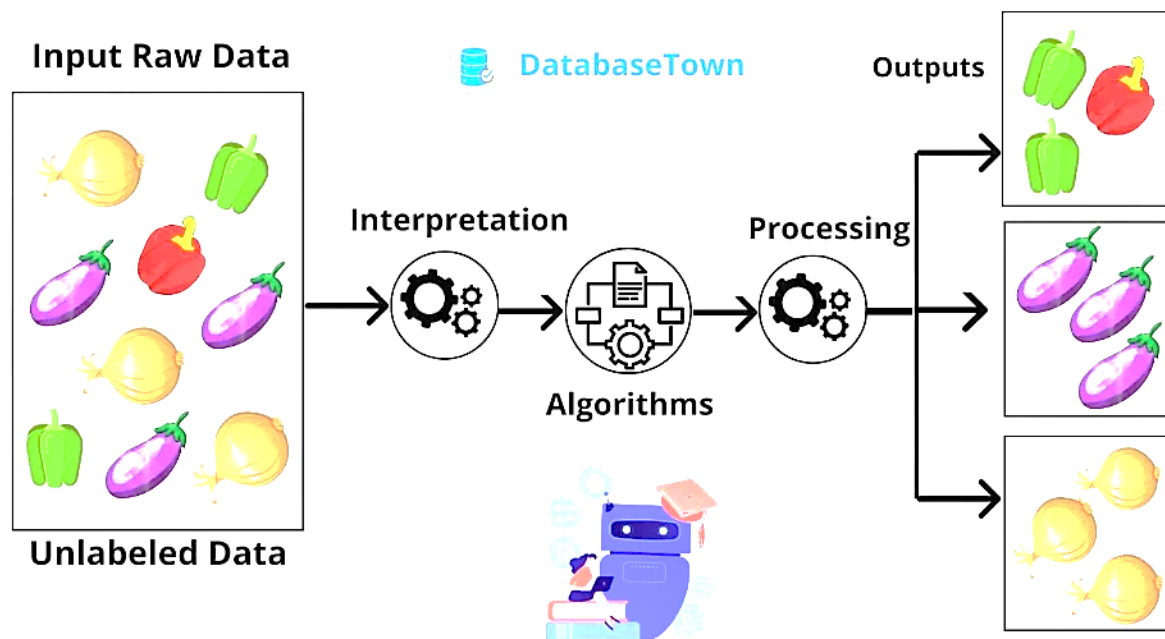
Develops a hierarchy of clusters by merging or splitting them based on similarity.

Agglomerative (Bottom-Up)

6. Start: every data point is its own cluster.
7. Find the two closest clusters and merge them.
8. Repeat until only one large cluster remains.

Divisive (Top-Down)

9. Start: all data points in one large cluster.
10. Split into smaller clusters based on differences.
11. Repeat until every point is its own cluster.



Unsupervised Learning

Figure 2.2 — Hierarchical Clustering: seven clusters merging step by step

2.2.4 K-Means Clustering

Groups data based on characteristics and similarities. K is the number of clusters chosen in advance.

- K = 3 means 3 clusters; K = 10 means 10 clusters.
- Each data point is assigned to the nearest cluster centre (centroid).
- Centroids are recalculated iteratively until clusters stabilise.

K-Means Objective Function:

$$J = \sum \sum \|x_i - \mu_k\|^2$$

Minimise the total squared distance of every point x_i from its cluster centroid μ_k .

K-Means Step-by-Step:

12. Choose K (number of clusters).
13. Initialise K centroids randomly.
14. Assign each data point to the nearest centroid.
15. Recalculate each centroid as the mean of its assigned points.

16. Repeat steps 3–4 until centroids stop changing (convergence).

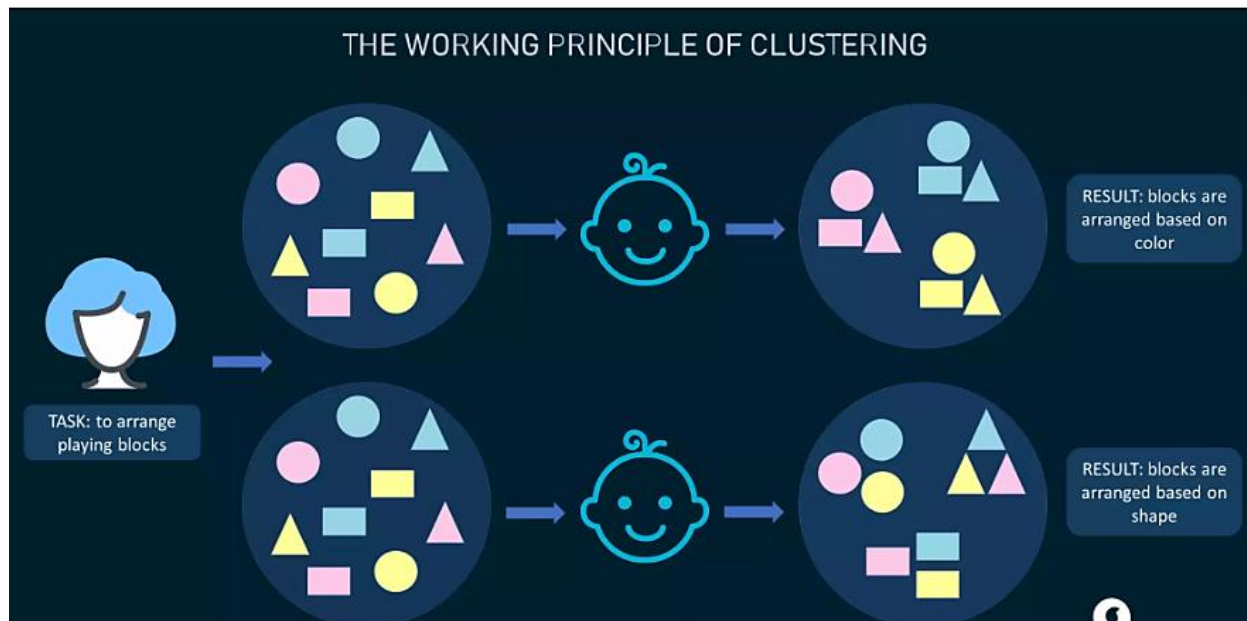


Figure 2.3 — K-Means: centroids (black dots) and cluster assignments

2.2.5 Fuzzy K-Means

Extension of K-Means for soft (overlapping) clustering. Each point gets a membership score for every cluster, measured by its distance from each centroid.

2.2.6 Gaussian Mixture Models (GMM)

Probabilistic clustering. Assumes data comes from a mixture of Gaussian (normal) distributions, one per cluster. Returns a probability (soft assignment) for each cluster instead of a hard label.

2.2.7 DBSCAN

Density-Based Spatial Clustering of Applications with Noise. Groups high-density regions and labels isolated points as noise (outliers). Does not require specifying K in advance.

- Core points: have many neighbours within a radius ϵ .
- Border points: near a core point but fewer neighbours.
- Noise points: isolated — do not belong to any cluster.

EXAM NOTE: Know all clustering types. Agglomerative vs. Divisive, K-Means vs. Fuzzy K-Means, and DBSCAN (density-based, handles noise) are frequently tested.

3. Association Rule Mining

3.1 Definition

Discovers interesting relationships or patterns between variables in large databases. Identifies sets of items that occur together.

3.2 Real-World Examples

- Bread buyers also tend to buy Butter / Jam.
- Diaper buyers also tend to buy Baby Wipes.

Rule format: IF customer buys X → THEN they will likely also buy Y.

3.3 Market Basket Analysis

Retailers analyse customer purchase data to identify products frequently bought together, then apply this to cross-selling and promotions.

3.4 Apriori Algorithm

17. Scan the database and count how often each item appears (support count).
18. Drop items below the minimum support threshold → frequent itemsets remain.
19. Combine frequent items into pairs, triplets, ... and keep those above minimum support.
20. Generate association rules from the frequent itemsets.
21. Filter rules by minimum confidence.

Term	Formula	Meaning
Support	$\text{count}(X \cup Y) / N$	How often X and Y appear together
Confidence	$\text{count}(X \cup Y) / \text{count}(X)$	Given X, how often is Y also bought?
Lift	$\text{Confidence} / \text{Support}(Y)$	How much more likely Y is when X is present

EXAM NOTE: Apriori = frequent itemsets + rule generation. Market basket analysis is the classic exam example.

4. Dimensionality Reduction

4.1 Definition

Reduces the number of input variables or features while retaining the most meaningful information.

4.2 Principal Component Analysis (PCA)

Why PCA is Needed:

- Highly correlated variables create redundancy in the data.
- Redundancy wastes computation and can hurt model accuracy.
- PCA compresses original variables into fewer new variables (Principal Components).

How PCA Works:

22. Identify correlated (redundant) features.
23. Transform original features into new axes called Principal Components (PCs).
24. PC_1 captures the most variance, PC_2 the second most, and so on.
25. Keep only the top PCs that explain most of the variance.
26. Discard the rest — dimensionality is reduced.

Example: Height and weight are highly correlated. PCA combines them into one principal component.

EXAM NOTE: PCA = reduce dimensions while preserving maximum variance. Used when features are correlated. Key terms: principal components, variance, redundancy.

5. Supervised vs. Unsupervised Learning

Feature	Supervised Learning	Unsupervised Learning
Data Type	Labeled data (input + output pairs)	Unlabeled data (input only)
Goal	Predict output for new inputs	Discover hidden patterns or structure
Human Involvement	Labels provided by humans	No labeling required
Output	Predefined categories or values	Clusters, rules, or components
Examples	Classification, Regression	Clustering, Association, PCA
Algorithms	Linear Regression, SVM, Decision Tree	K-Means, DBSCAN, Apriori, PCA
Evaluation	Accuracy, MSE, R^2	Silhouette score, Elbow method
When to Use	Known target variable exists	Exploring unknown data patterns

6. Reinforcement Learning

6.1 Definition

Reinforcement Learning (RL) enables an agent to learn in an interactive environment by trial and error using feedback from its own actions and experiences.

6.2 Core Concept

- Agents are self-trained using reward and punishment mechanisms.
- The agent takes actions and interacts with its environment.
- It is not trained on sample data — it learns by doing.
- A sequence of successful outcomes is reinforced to develop the best policy.

6.3 How RL Works (Step-by-Step)

27. Agent observes the current state of the environment.
28. Agent selects an action based on its current policy.
29. Environment transitions to a new state.
30. Environment returns a reward (positive) or penalty (negative).
31. Agent updates its policy to maximise future rewards.
32. Cycle repeats until the optimal strategy is learned.



Figure 6.1 — Reinforcement Learning: Agent-Environment Loop

6.4 The Dog and Stick Example

- 33. Master throws a stick.
- 34. Dog fetches the stick successfully.
- 35. Master gives the dog a bone (reward).
- 36. Dog learns: fetch stick = get bone.
- 37. Dog now fetches as fast as possible to maximise reward.

Dog = Agent. Master = Environment. Fetching = Action. Bone = Reward. 'Throw stick' = State signal. Fetching strategy = Policy.

Linear Regression Equation

$$Y = a + bx$$

$$a = \frac{[(\Sigma y)(\Sigma x^2) - (\Sigma x)(\Sigma xy)]}{[n(\Sigma x^2) - (\Sigma x)^2]}$$

$$b = \frac{[n(\Sigma xy) - (\Sigma x)(\Sigma y)]}{[n(\Sigma x^2) - (\Sigma x)^2]}$$

Figure 6.2 — Dog and Stick Reinforcement Learning Example

6.5 Important Terms in Reinforcement Learning

Term	Definition	Example (Dog Scenario)
Agent	The model being trained — the sole decision-maker and learner.	The dog
Environment	The world where the agent learns and decides actions.	Master and surroundings
Action	All possible steps the agent can take.	Fetch / ignore the stick

State	The current position or condition of the agent in the environment.	Stick is thrown
Reward	Scalar feedback from the environment (+ve or -ve).	Receiving a bone
Policy	Strategy mapping States to Actions — how the agent decides to act.	Always fetch when stick is thrown
Value	Expected future reward the agent receives by taking an action in a given state.	Expected bones over time

EXAM NOTE: Memorise all 7 RL terms. Draw the Agent–Environment loop. Know how reward and punishment drive learning.

7. Learning Scenarios — Exam Practice

Scenario	Type of ML	Reason
Facebook recognises your friend in a tagged photo album.	Supervised Learning	Tagged photos = labels. Model learns from labeled data.
Recommending new songs based on past music choices.	Supervised Learning	Model trains on existing genre labels/likes. Used by Netflix, Spotify, Pandora.
Analysing bank data to flag suspicious (fraud) transactions.	Unsupervised Learning	No 'fraud'/'not-fraud' labels exist. Model identifies outliers from patterns.

EXAM NOTE: Supervised = labels exist. Unsupervised = no labels, model finds patterns. Anomaly detection = unsupervised.

8. Linear Regression

8.1 Definition

A statistical method that predicts the value of an unknown dependent variable from one or more independent variables by fitting a straight line through the data.

8.2 Goal

Find the best values for the intercept and slope so the regression line minimises the difference (error) between predicted and actual values.

8.3 Sam's Height Prediction Example

Sam (age 10) wants to predict her adult height. She uses a dataset of parents' heights and their children's final heights.

- Input x: Parent's height
- Output y: Child's predicted height
- Linear regression finds the line that best predicts child height from parent height.

8.4 The Regression Line Equation

$$y = a + bx$$

Where:

- x — Independent Variable (plotted on X-axis): the input / predictor.
- y — Dependent Variable (plotted on Y-axis): the value being predicted.
- b — Slope: how much y changes for each unit increase in x.
- a — Y-Intercept: the value of y when x = 0.

Formulas for Slope (b) and Intercept (a):

$$b = \frac{n \cdot \Sigma xy - \Sigma x \cdot \Sigma y}{n \cdot \Sigma x^2 - (\Sigma x)^2}$$
$$a = \frac{\Sigma y - b \cdot \Sigma x}{n} = \frac{\Sigma y \Sigma x^2 - \Sigma x \Sigma xy}{n \Sigma x^2 - (\Sigma x)^2}$$

Worked Example:

Given: n = 4, $\Sigma x = 20$, $\Sigma y = 20$, $\Sigma x^2 = 124$, $\Sigma xy = 104$

$$a = [20(124) - 20(104)] / [4(124) - 400] = 400 / 96 = 4.17$$
$$b = [4(104) - 20(20)] / [4(124) - 400] = 16 / 96 = 0.166$$

Result: $y = 4.17 + 0.166x$



Figure 8.1 — Regression line: slope b and intercept a

8.5 Properties of Linear Regression

38. The line minimises the sum of squared differences between observed and predicted values.
39. The line always passes through the mean of X and the mean of Y .
40. The constant b_0 equals the y -intercept of the line.
41. The coefficient b_1 is the slope of the line.

8.6 Regression Coefficient

Using standard notation:

$$Y = B_0 + B_1X$$

Formula for B_1 (the regression coefficient):

$$B_1 = \frac{\sum[(x_i - \bar{x})(y_i - \bar{y})]}{\sum[(x_i - \bar{x})^2]}$$

Where x_i and y_i are observed data points, and $\bar{x}$, $\bar{y}$ are their means.

8.7 Applications of Linear Regression

- Market research and customer survey analysis.
- Studying engine performance in automobiles.
- Pricing decisions for goods.
- Astronomy — predicting celestial positions.
- House price prediction based on size, location, and age.

8.8 Types of Linear Regression

8.8.1 Simple Linear Regression

One independent variable, one dependent variable.

$$y = b_0 + b_1x$$

- b_0 = intercept (value of y when $x = 0$)
- b_1 = slope (change in y per unit change in x)

8.8.2 Multiple Linear Regression

Multiple independent variables, one dependent variable.

$$Y = b_0 + b_1X_1 + b_2X_2 + \dots + b_nX_n$$

- b_0 = intercept
- $b_1, b_2, \dots, b_n$ = slope coefficients for each variable $X_1, X_2, \dots, X_n$

Simple Linear Regression	Multiple Linear Regression
One independent variable (X)	Multiple independent variables ($X_1, X_2, \dots, X_n$)
$y = b_0 + b_1 \cdot X$	$Y = b_0 + b_1X_1 + b_2X_2 + \dots + b_nX_n$
Simple 2D relationship	Complex multi-factor relationship
Easy to visualise (2D plot)	Multi-dimensional — harder to visualise
e.g. Salary from experience	e.g. House price from size, location, age

8.9 Finding the Best Fit Line

Uses Ordinary Least Squares (OLS) — minimises the Residual Sum of Squares (RSS).

Residual:

The difference between the actual value and the predicted value for one data point.

Residual Sum of Squares (RSS):

$$RSS = \sum (y_i - \hat{y}_i)^2$$

OLS adjusts the intercept c and slope m to minimise RSS:

$$y = c + m \cdot x$$

What is Actually Happening?

Initially:

- line was completely wrong

After updates:

- line rotates and shifts
- gets closer to data points

Eventually it may become:

$$y = 2x$$

which perfectly fits this dataset.

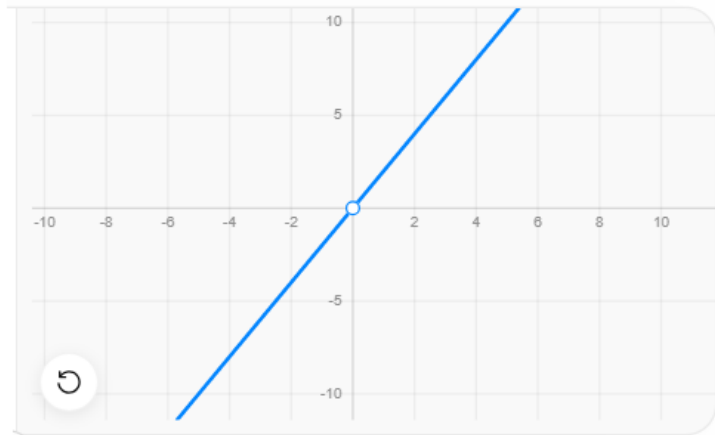


Figure 8.2 — Best Fit Line using Ordinary Least Squares

EXAM NOTE: OLS minimises RSS. Residual = actual – predicted. Best-fit line minimises total squared errors.

9. Cost Function

9.1 Definition

The cost function (also called loss function or objective function) measures how well the model is performing by calculating the difference between predicted and actual values.

9.2 Why Minimisation is Required

The goal is to find parameters that make predictions as accurate as possible. Minimising the cost function = finding those best parameters.

9.3 Mean Squared Error (MSE)

The most common cost function for regression:

$$MSE = \frac{1}{n} \cdot \Sigma (y_i - \hat{y}_i)^2$$

Where:

- n = total number of data points
- y_i = actual value of the i-th data point
- $\hat{y}_i$ = predicted value of the i-th data point

Lower MSE = better model. MSE penalises large errors more than small ones (squaring effect).

10. Model Evaluation Metrics

Example dataset used throughout this section (house price prediction):

- y (actual prices) = [100, 150, 200, 250, 300, 350, 400, 450, 500, 550]
- $\hat{y}$ (predicted prices) = [110, 140, 180, 240, 290, 320, 380, 420, 480, 520]
- $\bar{y}$ (mean of actual) = 325

10.1 SSR — Sum of Squares Regression

Variability in the dependent variable explained by the model.

$$SSR = \Sigma (\hat{y}_i - \bar{y})^2$$

Example result: SSR = 28,300

10.2 SSE — Sum of Squares Error

Variability not explained by the model — the remaining error.

$$SSE = \Sigma (y_i - \hat{y}_i)^2$$

Example result: SSE = 26,700

10.3 SST — Sum of Squares Total

Total variability in the dependent variable around its mean.

$$SST = \Sigma (y_i - \bar{y})^2$$

Example result: SST = 55,000

$$SST = SSR + SSE$$

10.4 MSE — Mean Squared Error

Average squared difference between predicted and actual values. Lower is better.

$$MSE = \frac{1}{n} \cdot \sum (y_i - \hat{y}_i)^2$$

Example result: MSE = 2,670

10.5 RMSE — Root Mean Squared Error

Square root of MSE. Expressed in the same units as the dependent variable — easier to interpret.

$$RMSE = \sqrt{MSE}$$

Example result: RMSE = $\sqrt{2670} \approx 51.68$

10.6 MAE — Mean Absolute Error

Average absolute difference between predicted and actual values. Less sensitive to outliers than MSE.

$$MAE = \frac{1}{n} \cdot \sum |y_i - \hat{y}_i|$$

Example result: MAE = 36

10.7 R² — Coefficient of Determination

Proportion of variance in the dependent variable explained by the model. Ranges from 0 to 1.

$$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

Example result: $R^2 = 28300 / 55000 = 0.5164 \rightarrow$ the model explains 51.64% of the variance.

$R^2 = 1 \rightarrow$ perfect model. $R^2 = 0 \rightarrow$ model explains nothing. Higher is better.

10.8 Summary Comparison Table

Metric	Formula	Example Value	Interpretation
SSR	$\sum (\hat{y}_i - \bar{y})^2$	28,300	Variance explained by model
SSE	$\sum (y_i - \hat{y}_i)^2$	26,700	Unexplained variance (error)
SST	$\sum (y_i - \bar{y})^2$	55,000	Total variance (SSR + SSE)
MSE	$(1/n) \cdot \sum (y_i - \hat{y}_i)^2$	2,670	Average squared error
RMSE	$\text{sqrt}(\text{MSE})$	51.68	Error in original units

MAE	$(1/n) \cdot \sum y_i - \hat{y}_i $	36	Average absolute error
R^2	SSR/SST	0.5164	% variance explained (0–1)

EXAM NOTE: SST = SSR + SSE. Higher R^2 = better. RMSE is in the same units as y. MAE is less sensitive to outliers than MSE.

11. Gradient Descent Algorithm

11.1 Definition

Gradient Descent is an optimisation algorithm that iteratively updates model parameters to minimise the cost function. It is the backbone of how machine learning models learn.

11.2 Why Gradient Descent is Needed

For a linear model:

$$y = a + bx$$

We need the best values of a (intercept) and b (slope) so the cost (MSE) is minimum. Gradient Descent finds them iteratively.

11.3 The Hill / Mountain Analogy

Imagine standing on a mountain and wanting to reach the lowest valley:

42. Look around — find the steepest downward direction.
43. Take a small step in that direction.
44. Repeat until you reach the bottom.

In Gradient Descent:

- Mountain surface = cost function landscape
- Lowest point (valley) = minimum cost → best parameters
- Step = one parameter update
- Step size = learning rate α

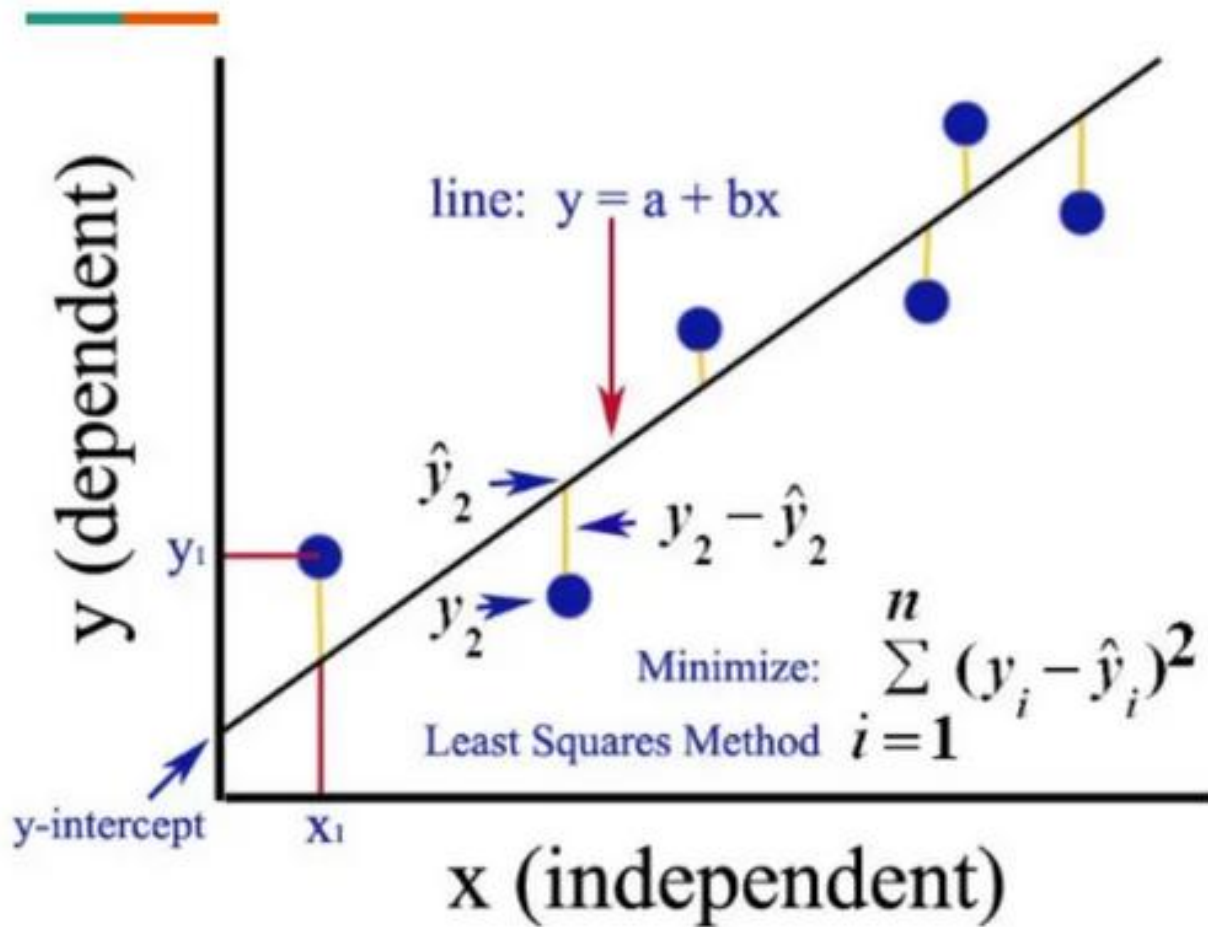


Figure 11.1 — Gradient Descent: moving toward the minimum of the cost function

11.4 Working Process (Step-by-Step)

45. Initialise w and b with random values (usually 0).
46. Predict: compute $\hat{y}$ using current w and b .
47. Calculate the cost (MSE between $\hat{y}$ and y).
48. Compute gradients $\partial J / \partial w$ and $\partial J / \partial b$.
49. Update w and b using the update rule.
50. Repeat steps 2–5 until cost converges (stops decreasing).

11.5 The Update Rule

$$w \leftarrow w - \alpha \cdot \frac{\partial J}{\partial w}$$

$$b \leftarrow b - \alpha \cdot \frac{\partial J}{\partial b}$$

Variable meanings:

- w — weight (slope parameter)

- b — bias (intercept parameter)
- α (alpha) — learning rate: controls step size
- $\partial J / \partial w$ — gradient of cost function w.r.t. w
- $\partial J / \partial b$ — gradient of cost function w.r.t. b

Gradient Formulas for Linear Regression:

$$\frac{\partial J}{\partial w} = \frac{2}{n} \cdot \sum x_i \cdot (\hat{y}_i - y_i)$$

$$\frac{\partial J}{\partial b} = \frac{2}{n} \cdot \sum (\hat{y}_i - y_i)$$

11.6 Learning Rate (α)

Small α (too small)	Large α (too large)
Very slow convergence	May overshoot the minimum
Takes many small steps	May fail to converge
More precise, less efficient	Less precise, faster initially
Risk: stuck in local minima longer	Risk: divergence (cost keeps increasing)

Regression Coefficient

Linear regression line, equation:

$$Y = B_0 + B_1 X$$

where,

B_0 is a Constant

B_1 is Regression Coefficient

Here, B_1 is the regression coefficient and its formula is,

$$B_1 = b_1 = \frac{\sum [(x_i - \bar{x})(y_i - \bar{y})]}{\sum [(x_i - \bar{x})^2]}$$

where,

→ x_i and y_i are Observed Data Sets

→ $\bar{x}$ and $\bar{y}$ are Mean Value

Figure 11.2 — Effect of learning rate on Gradient Descent convergence

Overshooting:

When α is too large, the update step jumps over the minimum. The cost oscillates or increases instead of decreasing.

Divergence:

When α is extremely large, the cost function value keeps increasing — the algorithm completely fails to converge.

EXAM NOTE: Too small α = slow. Too large α = overshoot / diverge. Typical starting values: 0.01, 0.001, 0.1.

11.7 Gradient Descent Applied to Linear Regression

Example: predicting house prices from house sizes. Start with $w = 0$, $b = 0$. Gradient Descent iteratively adjusts w and b to minimise MSE, converging to the optimal regression line.

11.8 Python Code Example

```
import numpy as np

# Dataset
x = np.array([1, 2, 3])
y = np.array([2, 4, 6])

# Initialise parameters
w = 0          # weight (slope)
b = 0          # bias (intercept)
alpha = 0.1    # learning rate

# Training loop - 5 iterations
for i in range(5):
    y_pred = w*x + b          # Step 1: Predict
    error = y_pred - y        # Step 2: Compute error

    dw = (2/len(x)) * np.sum(x * error) # Step 3a: Gradient for w
    db = (2/len(x)) * np.sum(error)      # Step 3b: Gradient for b

    w = w - alpha * dw          # Step 4a: Update w
    b = b - alpha * db          # Step 4b: Update b

    cost = np.mean(error**2)    # Step 5: Compute cost (MSE)
    print(f'Iter {i+1}: w={w:.4f}, b={b:.4f}, cost={cost:.4f}')
```

Code Line-by-Line Explanation:

- Dataset: $x=[1,2,3]$, $y=[2,4,6]$. True relationship is $y=2x \rightarrow$ optimal $w=2$, $b=0$.

- Init: $w=0$, $b=0$ — model starts with no knowledge.
- y_{pred} : current prediction using $y = w \cdot x + b$.
- error: difference between predicted and actual values.
- dw , db : gradients — direction to move parameters to reduce cost.
- w , b updates: move parameters toward lower cost.
- cost: MSE — should decrease each iteration, converging toward 0.

EXAM NOTE: Gradient Descent loop: Initialise → Predict → Error → Gradients → Update → Repeat. Know the update rule formulas.

12. Polynomial Regression

12.1 Definition

Polynomial Regression is used when the relationship between input and output is non-linear. It is an extension of linear regression that fits a curved line instead of a straight line.

12.2 Why Polynomial Regression — Non-linear Relationships

Age vs. Running Speed Example:

Age	Running Speed (km/h)
5	10
10	20
15	28
20	34
25	34
30	10

- Speed increases as people grow up, then decreases with age.
- This is NOT a straight-line relationship.
- A linear regression line fails here. Polynomial regression fits the curve correctly.

Polynomial Regression (Beginner Friendly Detailed Explanation)

Polynomial Regression is a type of regression used when the relationship between input and output is **not linear**.

It is an extension of **Linear Regression**.

Why Do We Need Polynomial Regression?

Suppose we want to predict:

- student performance
- population growth
- stock trends
- temperature variation
- disease spread

Sometimes data does NOT follow a straight line.

Figure 12.1 — Age vs Running Speed: non-linear relationship

12.3 The Polynomial Equation

General form:

$$y = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots + b_nx^n$$

Degree	Equation	Shape	Notes
1 (Linear)	$y = b_0 + b_1x$	Straight line	Simple linear regression
2 (Quadratic)	$y = b_0 + b_1x + b_2x^2$	Parabola	Most common polynomial
3 (Cubic)	$y = b_0 + b_1x + b_2x^2 + b_3x^3$	S-curve	More flexible
n (Higher)	$y = b_0 + b_1x + \dots + b_nx^n$	Complex curve	Risk of overfitting

12.4 Why It Is Still Called Linear

Although polynomial regression fits a curve, it remains a linear model because:

- It is linear in the parameters ($b_0, b_1, b_2, \dots, b_n$).
- The curve arises from the features x, x^2, x^3 — not from the coefficients.
- The model is still solved using OLS and Gradient Descent.

Linear means: the output is a linear combination of the parameters ($b_0, b_1, b_2, \dots$). Non-linearity comes from the feature terms ($x, x^2, x^3, \dots$), not the coefficients.

12.5 Cost Function and Gradient Descent in Polynomial Regression

Polynomial regression uses the same MSE cost function:

$$MSE = \frac{1}{n} \cdot \sum (y_i - \hat{y}_i)^2$$

Gradient Descent updates b_0, b_1, b_2 until cost is minimised. For a degree-2 polynomial:

$$\frac{\partial J}{\partial b_0} = -\frac{2}{n} \cdot \sum (y_i - \hat{y}_i)$$

$$\frac{\partial J}{\partial b_1} = -\frac{2}{n} \cdot \sum x_i \cdot (y_i - \hat{y}_i)$$

$$\frac{\partial J}{\partial b_2} = -\frac{2}{n} \cdot \sum x_i^2 \cdot (y_i - \hat{y}_i)$$

12.6 Exam Example — Student Scores

Study hours vs. exam score (non-linear increase):

Study Hours (x)	Exam Score (y)
-----------------	----------------

1	3
2	8
3	15
4	24
5	35

The pattern fits: $y = x^2 \rightarrow$ use degree-2 polynomial model.

12.7 Real-World Example — Salary Growth

Salary does not grow linearly with experience — it grows faster in early years. A polynomial model captures this curve better than a straight line.

EXAM NOTE: Polynomial regression = extend linear regression with x^2 , x^3 . Still minimises MSE using Gradient Descent. Degree 2 (quadratic) is most common.

13. Quick Revision Notes

13.1 Key Definitions

Term	One-Line Definition
Unsupervised Learning	ML on unlabeled data to find hidden patterns.
Clustering	Grouping data points based on similarity.
K-Means	Assigns data to K clusters using nearest centroid. Iterative.
Hierarchical Clustering	Agglomerative (bottom-up) or divisive (top-down) merging of clusters.
DBSCAN	Density-based clustering. Handles noise and outliers. No need to specify K.
Association Rule Mining	Finds items that co-occur frequently. Uses support, confidence, lift.
Apriori	Algorithm for frequent itemsets and association rule generation.
PCA	Reduces dimensions while preserving maximum variance.
Reinforcement Learning	Agent learns by trial and error using rewards and penalties.
Linear Regression	Fits $y = a + bx$ to predict a continuous output.
Cost Function (MSE)	Measures prediction error: $(1/n) \cdot \sum (y_i - \hat{y}_i)^2$
Gradient Descent	Optimisation algorithm: updates parameters to minimise cost iteratively.
Learning Rate (α)	Controls step size in gradient descent.
Polynomial Regression	Extends linear regression with higher-degree terms (x^2, x^3).
R^2 Score	Proportion of variance explained by the model. Range: 0 to 1.
OLS	Ordinary Least Squares — finds best-fit line by minimising RSS.

13.2 Important Formulas at a Glance

Formula	Name / Purpose
$y = a + bx$	Simple linear regression equation
$b = [n \cdot \sum xy - \sum x \cdot \sum y] / [n \cdot \sum x^2 - (\sum x)^2]$	Slope of regression line
$a = [\sum y - b \cdot \sum x] / n$	Intercept of regression line

$MSE = (1/n) \cdot \sum (y_i - \hat{y}_i)^2$	Mean Squared Error (cost function)
$RMSE = \sqrt{MSE}$	Root Mean Squared Error
$MAE = (1/n) \cdot \sum y_i - \hat{y}_i $	Mean Absolute Error
$SST = SSR + SSE$	Total = Explained + Error
$R^2 = SSR/SST = 1 - SSE/SST$	Coefficient of determination
$w \leftarrow w - \alpha \cdot (\partial J / \partial w)$	Gradient descent weight update
$b \leftarrow b - \alpha \cdot (\partial J / \partial b)$	Gradient descent bias update
$y = b_0 + b_1x + b_2x^2 + \dots$	Polynomial regression equation

13.3 Algorithm Summary

Algorithm	Type	Key Idea
K-Means	Clustering	Assign to nearest centroid, update centroids, repeat.
Fuzzy K-Means	Soft clustering	Each point has membership degree in multiple clusters.
GMM	Probabilistic clustering	Gaussian distributions model each cluster.
DBSCAN	Density clustering	Core / border / noise points. No need to specify K.
Hierarchical	Clustering	Agglomerative (merge) or divisive (split). Dendrogram output.
Apriori	Association mining	Frequent itemsets $\rightarrow$ rules. Uses support & confidence.
PCA	Dimensionality reduction	Project to principal components. Preserve variance.
Linear Regression	Supervised / Regression	Minimise MSE to find best-fit line.
Gradient Descent	Optimisation	Iterative parameter updates using partial derivatives.
Polynomial Regression	Supervised / Regression	Fit curves using higher-degree feature terms x^2, x^3 .

13.4 Key Differences to Remember

Comparison	Difference
Supervised vs Unsupervised	Supervised has labels; unsupervised does not.
Hard vs Soft Clustering	Hard: one cluster per point. Soft: multiple with membership degrees.

Agglomerative vs Divisive	Agglomerative merges up (bottom-up); divisive splits down (top-down).
K-Means vs DBSCAN	K-Means needs K specified; DBSCAN does not. DBSCAN handles noise.
MSE vs MAE	MSE penalises large errors more. MAE treats all errors equally.
RMSE vs MSE	RMSE is in original units. MSE is in squared units.
Simple vs Multiple Regression	Simple: 1 predictor. Multiple: many predictors.
Linear vs Polynomial	Linear: straight line. Polynomial: curve. Both minimise MSE.
Small vs Large α	Small: slow convergence. Large: may diverge.
SSR vs SSE vs SST	SSR = explained, SSE = error, SST = total. $SST = SSR + SSE$.

13.5 Top 15 Exam Questions

51. Define unsupervised learning and give two real-world applications.
52. Explain K-Means clustering step-by-step and state its objective function.
53. What is the difference between agglomerative and divisive hierarchical clustering?
54. Explain the Apriori algorithm using the bread-and-butter example.
55. What is PCA and why is it needed? Define 'principal component'.
56. Define all 7 RL terms: Agent, Environment, Action, State, Reward, Policy, Value.
57. Explain the Dog and Stick example in Reinforcement Learning.
58. Write the linear regression equation $y = a + bx$ and explain each variable.
59. What is OLS? What does it minimise?
60. Compare SSR, SSE, SST, MSE, RMSE, MAE, and R^2 with formulas.
61. Explain Gradient Descent using the mountain analogy.
62. What happens if the learning rate α is too large or too small?
63. Write and explain the Gradient Descent parameter update rule.
64. Explain why polynomial regression is still called a linear model.
65. Write the polynomial regression equation for degree 2 and degree 3.

— End of Study Notes —